

# 🔍 Complete Guide to Reducing Loss Function

## Quick Commands (Copy & Paste)

### Best Overall Performance

```
# Recommended: Focal Loss + 50 epochs + smaller learning rate
python train.py --epochs 50 --lr 0.0005 --loss focal --batch-size 16
```

### For Limited GPU Memory

```
python train.py --epochs 50 --lr 0.0005 --loss focal --batch-size 8
```

### Maximum Quality (Long Training)

```
python train.py --epochs 100 --lr 0.0003 --loss smooth --batch-size 32
```

---

## 📦 Loss Function Options

### 1. Cross Entropy (Default)

```
python train.py --epochs 30 --loss ce
```

- ☐ Standard and reliable
- ☐ Works well for balanced datasets
- ☐ May struggle with class imbalance

### 2. Focal Loss (Recommended for Medical Images)

```
python train.py --epochs 30 --loss focal
```

- ☐ **Better for rare classes**
- ☐ Focuses on hard-to-classify examples
- ☐ **Recommended for osteoporosis grading**
- ☐ Often achieves **10-20% lower loss** than CE

### 3. Label Smoothing (Better Generalization)

```
python train.py --epochs 30 --loss smooth
```

- ☐ Prevents overconfidence
- ☐ Better validation performance

- ☐ Reduces overfitting

---

## ☐ Strategies to Reduce Loss

### Strategy 1: More Training Time

```
# Start with 50 epochs
python train.py --epochs 50 --loss focal

# If still improving, try 100
python train.py --epochs 100 --loss focal
```

**Expected Impact:** 30-50% loss reduction compared to 30 epochs

### Strategy 2: Optimal Learning Rate

```
# Try different learning rates
python train.py --epochs 30 --lr 0.001      # Default
python train.py --epochs 30 --lr 0.0005    # Lower (more stable)
python train.py --epochs 30 --lr 0.0003    # Even lower (very stable)
python train.py --epochs 30 --lr 0.002    # Higher (faster but risky)
```

**Pro Tip:** Start with 0.0005 for medical images

### Strategy 3: Batch Size Optimization

```
# Larger batches = more stable gradients
python train.py --epochs 30 --batch-size 32 # If GPU allows
python train.py --epochs 30 --batch-size 24 # Medium
python train.py --epochs 30 --batch-size 16 # Default
python train.py --epochs 30 --batch-size 8  # If memory limited
```

### Strategy 4: Weight Decay (Regularization)

```
# Reduce overfitting
python train.py --epochs 30 --weight-decay 0.001 # Stronger regularization
python train.py --epochs 30 --weight-decay 0.0005 # Default
python train.py --epochs 30 --weight-decay 0.0001 # Lighter regularization
```

### Strategy 5: Image Size

```
# Larger images = more details = potentially better learning
python train.py --epochs 30 --img-size 800 # Larger (needs more GPU)
python train.py --epochs 30 --img-size 640 # Default (balanced)
python train.py --epochs 30 --img-size 512 # Smaller (faster)
```

---

## □ Recommended Configurations

### Configuration 1: Best Accuracy (Recommended)

```
python train.py \  
    --epochs 50 \  
    --lr 0.0005 \  
    --loss focal \  
    --batch-size 16 \  
    --img-size 640 \  
    --weight-decay 0.0005
```

**Expected:** ~60-70% validation accuracy, loss < 0.8

### Configuration 2: Fast Convergence

```
python train.py \  
    --epochs 30 \  
    --lr 0.001 \  
    --loss focal \  
    --batch-size 32 \  
    --img-size 512
```

**Expected:** ~55-65% validation accuracy, loss ~1.0

### Configuration 3: Maximum Quality (Long Training)

```
python train.py \  
    --epochs 100 \  
    --lr 0.0003 \  
    --loss smooth \  
    --batch-size 24 \  
    --img-size 800 \  
    --weight-decay 0.001
```

**Expected:** ~70-80% validation accuracy, loss < 0.6

### Configuration 4: Limited GPU Memory

```
python train.py \  
    --epochs 50 \  
    --lr 0.0005 \  
    --loss focal \  
    --batch-size 4 \  
    --img-size 512 \  
    --workers 2
```

---

## □ Understanding Loss Values

## What's a Good Loss?

Loss > 2.0    ☐ Model not learning (check data/hyperparameters)  
Loss 1.5-2.0   ☐ Early training or poor hyperparameters  
Loss 1.0-1.5   ✓ Decent performance  
Loss 0.6-1.0   ✓✓ Good performance  
Loss 0.3-0.6   ✓✓✓ Very good performance  
Loss < 0.3    ☐ Possible overfitting (check validation loss)

## Training vs Validation Loss

Train Loss < Val Loss (normal)    ✓ Model generalizing well  
Train Loss ≈ Val Loss            ✓✓ Perfect scenario  
Train Loss >> Val Loss (big gap)   ☐ Overfitting - need regularization  
Train Loss > Val Loss            ☐ Unusual - check data

---

## ☐ Advanced Techniques

### 1. Learning Rate Schedule Fine-Tuning

Modify in train.py:

```
# Cosine Annealing (default)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)

# Step LR (drops at intervals)
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.5)

# Reduce on Plateau (adaptive)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min',
                                                    factor=0.5, patience=5)
```

### 2. Gradient Clipping

Add to train.py in the training loop:

```
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0) # Prevents
exploding gradients
optimizer.step()
```

### 3. Mixed Precision Training (Faster + Lower Memory)

```
from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler()

with autocast():
```

```
outputs = model(images)
loss = criterion(outputs, labels)

scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()
```

## 4. Data Augmentation Tuning

In train.py, modify augmentation strength:

```
train_transform = transforms.Compose([
    transforms.Resize((img_size, img_size)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15), # Increase from 10
    transforms.ColorJitter(brightness=0.3, contrast=0.3), # Increase from
0.2
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)), # Add
translation
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])
```

---

## □ Experiment Tracking

### Run Multiple Experiments

```
# Experiment 1: Baseline
python train.py --epochs 30 --save-dir runs/exp1_baseline

# Experiment 2: Focal Loss
python train.py --epochs 30 --loss focal --save-dir runs/exp2_focal

# Experiment 3: More Epochs
python train.py --epochs 50 --loss focal --save-dir runs/exp3_50epochs

# Experiment 4: Lower LR
python train.py --epochs 50 --lr 0.0003 --loss focal --save-dir
runs/exp4_lowlr

# Experiment 5: Best Config
python train.py --epochs 100 --lr 0.0003 --loss smooth --batch-size 24 --
save-dir runs/exp5_best
```

### Compare Results

After training, compare the training\_curves.png files in each runs/exp\_X/ folder.

---

## ❑ Quick Troubleshooting

### Loss Not Decreasing

```
# Try: Lower learning rate + Focal Loss
python train.py --epochs 50 --lr 0.0003 --loss focal
```

### Loss Decreasing Too Slowly

```
# Try: Higher learning rate + More epochs
python train.py --epochs 50 --lr 0.002 --loss focal
```

### Loss Jumps Around

```
# Try: Lower learning rate + Larger batch size
python train.py --epochs 30 --lr 0.0003 --batch-size 32
```

### Training Loss Low, Validation Loss High (Overfitting)

```
# Try: More regularization + Data augmentation
python train.py --epochs 30 --weight-decay 0.002 --loss smooth
```

### Both Losses High After Many Epochs

```
# Try: Check your data labels and increase model capacity
# Or use a pretrained model (transfer learning)
```

---

## ❑ Monitoring Training

Watch for these patterns during training:

### Good Training:

```
Epoch 1:  Train Loss: 1.50, Val Loss: 1.45  ✓
Epoch 10: Train Loss: 0.95, Val Loss: 1.00  ✓
Epoch 20: Train Loss: 0.70, Val Loss: 0.78  ✓
Epoch 30: Train Loss: 0.55, Val Loss: 0.65  ✓✓ (steady improvement)
```

### Overfitting:

```
Epoch 1:  Train Loss: 1.50, Val Loss: 1.45  ✓
Epoch 10: Train Loss: 0.95, Val Loss: 1.00  ✓
Epoch 20: Train Loss: 0.40, Val Loss: 0.95  ❑ (gap widening)
Epoch 30: Train Loss: 0.15, Val Loss: 1.20  ❑ (overfitting!)
```

### Not Learning:

```
Epoch 1: Train Loss: 1.79, Val Loss: 1.78  
Epoch 10: Train Loss: 1.78, Val Loss: 1.77  
Epoch 20: Train Loss: 1.77, Val Loss: 1.76  
Epoch 30: Train Loss: 1.76, Val Loss: 1.75 □ (stuck!)
```

---

## □ Final Recommendation

For hip X-ray Singh Index classification, use:

```
python train.py --epochs 50 --lr 0.0005 --loss focal --batch-size 16
```

This typically achieves:

- Training loss: 0.5-0.7
- Validation loss: 0.7-0.9
- Accuracy: 70-85%

**Then fine-tune with:**

```
python train.py --epochs 100 --lr 0.0003 --loss smooth --batch-size 16
```

Good luck! □